

[Marcar como hecha](#)

Observer

Es un patrón de diseño de comportamiento que te permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

Es muy utilizado en el modelo MVC

Explicado mejor: El patrón de diseño Observer tiene como objetivo modelizar una dependencia entre un sujeto y los observadores de modo que cada modificación del estado interno del sujeto sea notificada a sus observadores para que puedan actualizar su estado.

Un cliente puede visitar la tienda cada día para comprobar la disponibilidad del producto. Pero, mientras el producto está en camino, la mayoría de estos viajes serán en vano.

El objeto que tiene un estado interesante suele denominarse *sujeto*, pero, como también va a notificar a otros objetos los cambios en su estado, le llamaremos *notificador*.

Cuando le sucede un evento importante al notificador, recorre sus suscriptores y llama al método de notificación específico de sus objetos.

Utiliza el patrón Observer cuando los cambios en el estado de un objeto puedan necesitar cambiar otros objetos y el grupo de objetos sea desconocido de antemano o cambie dinámicamente.

Puedes experimentar este problema a menudo al trabajar con clases de la interfaz gráfica de usuario.

Utiliza el patrón cuando algunos objetos de tu aplicación deban observar a otros, pero sólo durante un tiempo limitado o en casos específicos. La lista de suscripción es dinámica, por lo que los suscriptores pueden unirse o abandonar la lista cuando lo deseen.

La meta del patrón *Observer* es establecer conexiones dinámicas de un único sentido entre objetos, donde algunos objetos actúan como subordinados de otros.

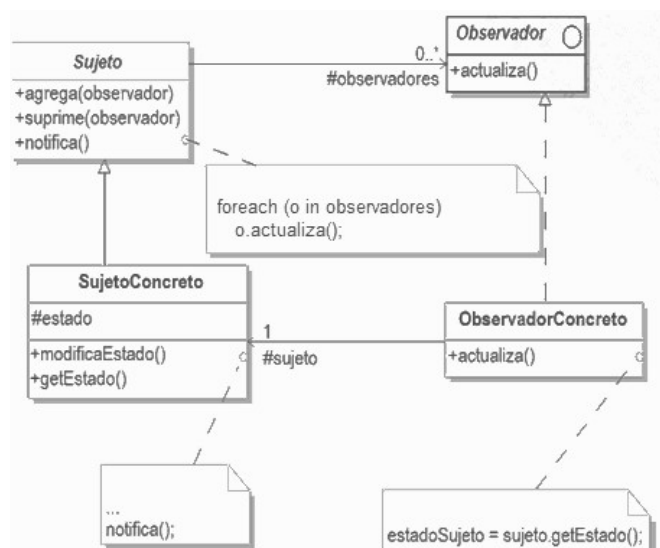
Dentro de la aplicación de los coches

Un diagrama de este patrón contendría las cuatro clases o interfaces siguientes:

- Sujeto es la clase abstracta que incluye todo objeto que notifica a los demás objetos de las modificaciones en su estado interno.
- Vehículo es la subclase concreta de Sujeto que describe a los vehículos. Gestiona dos atributos: descripción y precio.
- Observador es la interfaz de todo objeto que necesite recibir las notificaciones de cambio de estado provenientes de los objetos a los que se ha inscrito previamente.
- VistaVehiculo es la subclase concreta correspondiente a la implementación de Observador cuyas instancias muestran la información de un vehículo.

El funcionamiento es el siguiente: cada nueva vista se inscribe como observador de su vehículo mediante el método `agrega`. Cada vez que la descripción o el precio se actualizan, se invoca el método `notifica`. Este solicita a todos los observadores que se actualicen invocando a su método `actualiza`. En la clase `VistaVehiculo`, este último método se llama `redibuja`.

La solución implementada mediante el patrón de diseño Observer es genérica. En efecto, todo el mecanismo de observación está implementado en la clase `Sujeto` y en la interfaz `Observador`, que pueden tener otras subclases distintas de `Vehiculo` y `VistaVehiculo`.



Los participantes del patrón de diseño Observer son los siguientes:

- **Sujeto**(**AbstractSujeto**) es la clase abstracta que incluye la asociación con los observadores así como los métodos para agregar o suprimir observadores.
- **Observador**(**ObservadorInterfaz**) es la interfaz que es necesario implementar para recibir las notificaciones (método `actualiza`).
- **SujetoConcreto** (**Vehiculo**) es una clase correspondiente a la implementación de un sujeto. Un sujeto envía una notificación cuando su estado interno se ha modificado.
- **ObservadorConcreto** (**VistaVehiculo**) es una clase de implementación de un observador. Mantiene una referencia hacia el sujeto e implementa el método `actualiza`.

El patrón de diseño Observer se utiliza en los casos siguientes:

- Una modificación en el estado de un objeto genera modificaciones en otros objetos que se determinan dinámicamente.
- Un objeto quiere avisar a otros objetos sin tener que conocer su tipo, es decir sin estar fuertemente acoplado a ellos.
- No se desea fusionar dos objetos en uno solo.

Ejemplo de código

```
<?php
abstract class AbstractSujeto
{
    protected array $observadores = [];

    public function agrega(ObservadorInterfaz $observador): void
    {
        $this->observadores[] = $observador;
    }

    public function retira(ObservadorInterfaz $observador): void
    {
        $key = array_search($observador, $this->observadores);

        if (false !== $key) {
            unset($this->observadores[$key]);
        }
    }

    public function notifica(): void
    {
        foreach ($this->observadores as $observador) {
            $observador->actualiza();
        }
    }
}
```

El código fuente de la interfaz ObservadorInterfaz es muy simple puesto que solo contiene la firma del método actualiza.

```
<?php

interface ObservadorInterfaz
{
    public function actualiza(): void;
}
```

El código fuente de la clase Vehiculo aparece a continuación. Contiene dos atributos y los getter y setter para ambos atributos. Los dos setter invocan al método notifica.

```
<?php

include ('AbstractSujeto.php');

class Vehiculo extends AbstractSujeto
{
    protected string $descripcion;

    protected float $precio;

    public function getDescripcion(): string
    {
        return $this->descripcion;
    }

    public function getPrecio(): float
    {
        return $this->precio;
    }

    public function setDescripcion(string $descripcion): void
    {
        $this->descripcion = $descripcion;
        $this->notifica();
    }

    public function setPrecio(float $precio): void
    {
        $this->precio = $precio;
        $this->notifica();
    }
}
```

La clase VistaVehiculo gestiona un texto que contiene la descripción y el precio del vehículo asociado (el asunto). Este texto se actualiza tras cada notificación en el cuerpo del método actualiza. El método redibuja imprime este texto por pantalla.

```

<?php

include ('ObservadorInterfaz.php');

class VistaVehiculo implements ObservadorInterfaz
{
    protected Vehiculo $vehiculo;

    protected string $texto;

    protected string $nombreVista;

    public function __construct(string $nombreVista, Vehiculo $vehiculo)
    {
        $this->nombreVista = $nombreVista;
        $this->vehiculo = $vehiculo;
        $vehiculo->agrega($this);
        $this->actualizaTexto();
    }

    protected function actualizaTexto(): void
    {
        $precio = number_format($this->vehiculo->getPrecio(), 2, ',', ' ');
        $descripcion = $this->vehiculo->getDescripcion();
        $this->texto = $this->nombreVista . ' : ' . $descripcion . ' ',
Precio: ' . $precio
. "<br>";

    }

    public function actualiza(): void
    {
        $this->actualizaTexto();
        $this->redibuja();
    }

    public function redibuja(): void
    {
        echo $this->texto;
    }
}

```

Por último, aquí tenemos el programa que realiza la llamada. Este programa crea un vehículo y a continuación una vista a la que le pide la visualización. A continuación se modifica el precio y la vista se refresca. A continuación se crea una segunda vista que se asocia al mismo vehículo. El precio se modifica de nuevo y ambas vistas se refrescan.

```

<?php

// include ('AbstractSujeto.php');
// include ('ObservadorInterfaz.php');
include ('Vehiculo.php');
include ('VistaVehiculo.php');

$vehiculo = new Vehiculo();
$vehiculo->setDescripcion('Vehículo económico');
$vehiculo->setPrecio(4999.99);

$vistaVehiculo = new VistaVehiculo('Observador 1', $vehiculo);
$vistaVehiculo->redibuja();
$vehiculo->setPrecio(4500);

$vistaVehiculo2 = new VistaVehiculo('Observador 2', $vehiculo);
$vehiculo->setPrecio(5500);

$vehiculo->retira($vistaVehiculo);
$vehiculo->setPrecio(6500);

```

?

El resultado de la ejecución de este programa es el siguiente.

```
Observador 1: "Vehículo económico", Precio: 4999,99
Observador 1: "Vehículo económico", Precio: 4500,00
Observador 1: "Vehículo económico", Precio: 5500,00
Observador 2: "Vehículo económico", Precio: 5500,00
Observador 2: "Vehículo económico", Precio: 6500,00
```

Ejemplo segundo de código

```
<?php
```

```
/**
 * PHP tiene un par de interfaces integradas relacionadas con el patrón Observer.
 * Este es el aspecto de la interfaz Asunto:

 * @link http://php.net/manual/en/class.splsubject.php
 *
 * interface SplSubject
 * {
 *
 *     // Attach an observer to the subject.
 *     public function attach(SplObserver $observer);
 *
 *     // Detach an observer from the subject.
 *     public function detach(SplObserver $observer);
 *
 *     // Notify all observers about an event.
 *     public function notify();
 *
 * }
 *
 * También hay una interfaz integrada para los observadores:
 *
 * @link http://php.net/manual/en/class.splobserver.php
 *
 * interface SplObserver
 * {
 *
 *     public function update(SplSubject $subject);
 *
 * }
 */

/**
 * El Sujeto posee algún estado importante y notifica a los observadores cuando el estado
 * cambia.
 */
class Subject implements \SplSubject
{
    /**
     * @var int En aras de la simplicidad, el estado del sujeto, esencial para
     * todos los suscriptores, se almacena en esta variable.
     */
    public $state;

    /**
     * @var \SplObjectStorage Lista de abonados. En la vida real, la lista de
     * suscriptores puede ser almacenada de manera más completa (categorizada por evento
     * tipo, etc.).
     */
    private $observers;
```

?

```
public function __construct()
{
    $this->observers = new \SplObjectStorage();
} /**
 * Los métodos de gestión de las suscripciones
 */
public function attach(\SplObserver $observer): void
{
    echo "Asunto: Adjuntar un observador.<br>";
    $this->observers->attach($observer);
}
public function detach(\SplObserver $observer): void
{
    $this->observers->detach($observer);
    echo "Asunto: desvincular un observador.<br>";
}/**
 * Activar una actualización en cada suscriptor.
 */
public function notify(): void
{
    echo "asunto: notificando a observadores...<br>";
    foreach ($this->observers as $observer) {
        $observer->update($this);
    }
} /**
 * Por lo general, la lógica de suscripción es sólo una fracción de lo que un sujeto puede
 * hacer realmente. Los sujetos comúnmente contienen alguna lógica de negocio importante, que
 * desencadena un método de notificación cuando algo importante está a punto de
 * ocurrir (o después de ello).
 */
public function someBusinessLogic(): void
{
    echo "<br> asunto: estoy haciendo algo que merece la pena, te interesará.<br>";
    $this->state = rand(0, 10);

    echo "asunto: mi estado ha cambiado a : {$this->state}<br>";
    $this->notify();
}
}
/**
 * Los Observadores concretos reaccionan a las actualizaciones emitidas por el Sujeto al que han sido
 * adjuntado.
 */
class ConcreteObserverA implements \SplObserver
{
    public function update(\SplSubject $subject): void
    {
        if ($subject->state < 3) {
            echo "Observadorconcreto A: Reacciona al evento.<br>";
        }
    }
}
class ConcreteObserverB implements \SplObserver
{
    public function update(\SplSubject $subject): void
```

?

```
{
    if ($subject->state == 0 || $subject->state >= 2) {
        echo "Observadorconcreto b: Reacciona al evento.<br>";
    }
}

/**
 * Código del cliente
 */
$subject = new Subject();
$o1 = new ConcreteObserverA();
$subject->attach($o1);
$o2 = new ConcreteObserverB();
$subject->attach($o2);
$subject->someBusinessLogic();
$subject->someBusinessLogic();
$subject->detach($o2);
$subject->someBusinessLogic();

?>
```